# Documentation technique du wrapper GTK4 (Volume 2)

Projet GTK Wrapper

8 mars 2026

## Table des matières

**1 Introduction** **3**

**2 Module `window`** **4**
2.1 Macros de Configuration . . . . . 4
2.2 Fonctions API . . . . . 4

**3 Module `button`** **6**
3.1 Macros de Configuration . . . . . 6
3.2 Fonctions API . . . . . 6

**4 Module `common`** **8**
4.1 Macros de Configuration . . . . . 8
4.2 Fonctions API . . . . . 8

**5 Module `container`** **9**
5.1 Macros de Configuration . . . . . 9
5.2 Fonctions API . . . . . 9

**6 Module `display`** **11**
6.1 Macros de Configuration . . . . . 11
6.2 Fonctions API . . . . . 12

**7 Module `input`** **13**
7.1 Macros de Configuration . . . . . 13
7.2 Fonctions API . . . . . 14

**8 Module `toggle`** **15**
8.1 Macros de Configuration . . . . . 15
8.2 Fonctions API . . . . . 15

**9 Module `date`** **17**
9.1 Macros de Configuration . . . . . 17
9.2 Fonctions API . . . . . 17

**10 Module `menu`** **19**
10.1 Macros de Configuration . . . . . 19
10.2 Fonctions API . . . . . 19

**11 Module `dialog`** **21**
11.1 Macros de Configuration . . . . . 21
11.2 Fonctions API . . . . . 21

**12 Module `separator`** **23**
12.1 Macros de Configuration . . . . . 23
12.2 Fonctions API . . . . . 23

**13 Module `theme`** **24**
13.1 Macros de Configuration . . . . . 24
13.2 Fonctions API . . . . . 24

**14 Module `demo_app`** **25**
14.1 Macros de Configuration . . . . . 25
14.2 Fonctions API . . . . . 25

# 1 Introduction

Ce volume décrit l'implémentation des modules widgets du projet GTK4. Pour chaque module, on présente la configuration publique (macros et structures) puis la logique des fonctions API côté `.c`. L'objectif est de relier l'interface de programmation aux choix techniques de robustesse (valeurs par défaut, vérifications `NULL`, branchement des signaux).

# 2 Module `window`

Ce module crée la fenêtre principale GTK et applique les propriétés de base (titre, taille, redimensionnement, modalité et style).

## 2.1 Macros de Configuration

**Macro `WINDOW_CONFIG_DEFAULT`**

```c
#define WINDOW_CONFIG_DEFAULT ((window_config){ \
    .title = NULL, \
    .icon_name = NULL, \
    .widget_name = NULL, \
    .background_color = NULL, \
    .background_image_path = NULL, \
    .default_width = -1, \
    .default_height = -1, \
    .min_width = 0, \
    .min_height = 0, \
    .max_width = 0, \
    .max_height = 0, \
    .resizable = true, \
    .decorated = true, \
    .modal = false, \
    .maximized = false, \
    .present_on_create = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})
```

Cette macro fournit une configuration sûre avec des pointeurs initialisés à `NULL` et des booléens cohérents. Elle évite de lire des champs indéfinis au moment de la création de la fenêtre. En pratique, on part de cette base puis on surcharge uniquement les options nécessaires.

Cette initialisation par défaut est importante en C : sans valeurs explicites, certains champs peuvent contenir des données aléatoires en mémoire. En utilisant cette macro, on réduit les comportements non déterministes et les erreurs difficiles à reproduire.

## 2.2 Fonctions API

```c
GtkWidget *create_window(GtkApplication *app, const window_config *config) {
    GtkWidget *window = gtk_application_window_new(app);
    if (config == NULL) {
        return window;
    }

    if (config->title) {
        gtk_window_set_title(GTK_WINDOW(window), config->title);
    }
    if (config->icon_name) {
```

```
        gtk_window_set_icon_name(GTK_WINDOW(window), config->icon_name);
    }

    gtk_window_set_default_size(GTK_WINDOW(window), config->default_width,
        config->default_height);
    gtk_window_set_resizable(GTK_WINDOW(window), config->resizable);
    gtk_window_set_decorated(GTK_WINDOW(window), config->decorated);
    gtk_window_set_modal(GTK_WINDOW(window), config->modal);

    apply_window_background(config, window);
    apply_widget_style(window, &config->style);

    if (config->maximized) {
        gtk_window_maximize(GTK_WINDOW(window));
    }
    if (config->present_on_create) {
        gtk_window_present(GTK_WINDOW(window));
    }

    return window;
}
```

La fonction applique une stratégie défensive : toujours retourner un widget valide, puis enrichir selon la configuration. Cela simplifie l'appelant et réduit les cas d'erreur côté application.

Le rôle de `create_window` est aussi d'unifier la politique d'initialisation de la fenêtre : dimensions, état de présentation et style CSS sont traités au même endroit. Cette centralisation améliore la maintenabilité et évite de disperser des appels GTK critiques dans plusieurs fichiers.

# 3 Module button

Le module button encapsule la création des boutons d'action et des boutons radio, avec options de style, icône et callbacks.

## 3.1 Macros de Configuration

**Macros BUTTON_CONFIG_DEFAULT et RADIO_BUTTON_CONFIG_DEFAULT**

```c
#define BUTTON_CONFIG_DEFAULT ((button_config){ \
    .label = NULL, \
    .icon_name = NULL, \
    .theme_variant = NULL, \
    .use_underline = false, \
    .has_frame = true, \
    .can_shrink = true, \
    .icon_size = 0, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_clicked = NULL, \
    .user_data = NULL, \
})

#define RADIO_BUTTON_CONFIG_DEFAULT ((radio_button_config){ \
    .label = NULL, \
    .group_with = NULL, \
    .is_active = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_toggled = NULL, \
    .user_data = NULL, \
})
```

Ces macros initialisent toutes les options dans un état prévisible. Les pointeurs NULL évitent les accès mémoire invalides, et les callbacks nuls empêchent les appels de fonction indéfinis. Cela réduit fortement les risques de segmentation fault dans les chemins de création.

L'intérêt pratique est de pouvoir écrire un code d'assemblage très compact : on initialise avec la macro, puis on personnalise seulement le label, l'icône et le callback. Cette méthode diminue les oublis de champs et stabilise l'interface.

## 3.2 Fonctions API

```c
GtkWidget *create_button(const button_config *config) {
    GtkWidget *button;
    const char *label;

    if (config == NULL) {
        return gtk_button_new();
    }

```

```
    label = config->label ? config->label : "";
    button = config->use_underline ? gtk_button_new_with_mnemonic(label)
                                   : gtk_button_new_with_label(label);

    gtk_button_set_has_frame(GTK_BUTTON(button), config->has_frame);
    gtk_button_set_can_shrink(GTK_BUTTON(button), config->can_shrink);

    apply_theme_variant(button, config->theme_variant);
    apply_widget_style(button, &config->style);

    if (config->on_clicked) {
        g_signal_connect(button, "clicked", G_CALLBACK(config->on_clicked),
            config->user_data);
    }

    return button;
}
```

La fonction construit un bouton utilisable même en cas de configuration partielle. Elle ajoute ensuite les options visuelles et connecte le callback si fourni.

La fonction `create_button` joue aussi un rôle d'adaptation entre des besoins métiers simples et l'API GTK plus verbeuse. En encapsulant les variantes de thème et le branchement du signal `clicked`, elle rend le code applicatif plus lisible.

# 4 Module common

Le module `common` centralise l'application des styles transverses à tous les widgets (CSS, marges, alignements, expansion, visibilité, infobulles).

## 4.1 Macros de Configuration

**Macro `WIDGET_STYLE_CONFIG_DEFAULT`**

```
#define WIDGET_STYLE_CONFIG_DEFAULT ((widget_style_config){0})
```

Cette macro force une initialisation complète à zéro de la structure de style. En C, cela signifie des pointeurs à `NULL` et des flags booléens à `false`. Ce choix évite les lectures de mémoire non initialisée et réduit les plantages aléatoires.

Cette convention est essentielle pour éviter des erreurs de segmentation quand des champs optionnels ne sont pas renseignés. Elle fournit un contrat clair : tout champ non modifié reste dans un état neutre.

## 4.2 Fonctions API

```
void apply_widget_style(GtkWidget *widget, const widget_style_config *style) {
    if (widget == NULL || style == NULL) {
        return;
    }

    if (style->css_class && style->css_class[0] != '\0') {
        gtk_widget_add_css_class(widget, style->css_class);
    }

    gtk_widget_set_margin_top(widget, style->margin_top);
    gtk_widget_set_margin_bottom(widget, style->margin_bottom);
    gtk_widget_set_margin_start(widget, style->margin_start);
    gtk_widget_set_margin_end(widget, style->margin_end);

    if (style->set_halign) gtk_widget_set_halign(widget, style->halign);
    if (style->set_valign) gtk_widget_set_valign(widget, style->valign);
    if (style->set_hexpand) gtk_widget_set_hexpand(widget, style->hexpand);
    if (style->set_vexpand) gtk_widget_set_vexpand(widget, style->vexpand);
}
```

Cette fonction sert d'adaptateur entre la structure de configuration et l'API GTK. Elle applique uniquement les champs explicitement demandés pour préserver le comportement attendu des widgets.

Sur le plan d'architecture, `apply_widget_style` factorise la logique transversale pour tous les composants. Cela réduit la duplication de code et garantit une cohérence visuelle globale dans l'application.

# 5 Module container

Le module container regroupe les conteneurs de mise en page : grille, boîte linéaire et pile de pages.

## 5.1 Macros de Configuration

Macros GRID_CONFIG_DEFAULT, BOX_CONFIG_DEFAULT, STACK_CONFIG_DEFAULT

```
#define GRID_CONFIG_DEFAULT ((grid_config){ \
    .column_spacing = 0, \
    .row_spacing = 0, \
    .homogeneous_columns = false, \
    .homogeneous_rows = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})

#define BOX_CONFIG_DEFAULT ((box_config){ \
    .orientation = GTK_ORIENTATION_VERTICAL, \
    .spacing = 0, \
    .homogeneous = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})

#define STACK_CONFIG_DEFAULT ((stack_config){ \
    .transition = GTK_STACK_TRANSITION_TYPE_NONE, \
    .duration_ms = 200, \
    .vhomogeneous = false, \
    .hhomogeneous = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})
```

Ces macros garantissent des configurations minimales sûres et prévisibles. Elles évitent de laisser des paramètres de layout dans un état indéfini, ce qui simplifie le débogage de l'interface. Elles permettent aussi une initialisation rapide avant surcharge locale.

En pratique, ces valeurs par défaut évitent les erreurs de mise en page (espacements incohérents, transition non souhaitée, orientation oubliée). Elles sécurisent le démarrage de l'interface dès les premiers prototypes.

## 5.2 Fonctions API

```
GtkWidget *create_grid(const grid_config *config) {
    GtkWidget *grid = gtk_grid_new();
    if (config == NULL) return grid;

    gtk_grid_set_column_spacing(GTK_GRID(grid), config->column_spacing);
    gtk_grid_set_row_spacing(GTK_GRID(grid), config->row_spacing);
    gtk_grid_set_column_homogeneous(GTK_GRID(grid), config->homogeneous_columns)
        ;
```

```
    gtk_grid_set_row_homogeneous(GTK_GRID(grid), config->homogeneous_rows);
    apply_widget_style(grid, &config->style);
    return grid;
}

GtkWidget *create_stack(const stack_config *config) {
    GtkWidget *stack = gtk_stack_new();
    if (config == NULL) return stack;

    gtk_stack_set_transition_type(GTK_STACK(stack), config->transition);
    gtk_stack_set_transition_duration(GTK_STACK(stack), config->duration_ms);
    return stack;
}
```

La logique applique les propriétés seulement si `config` existe, ce qui protège le code appelant et facilite les usages simples.

Le module `container` joue un rôle clé dans l'ergonomie globale : il encapsule la disposition des widgets et isole les détails GTK liés au layout. Cette séparation améliore la lisibilité du code applicatif.

# 6 Module display

Le module display gère les widgets d'affichage : progression, indicateur d'activité, texte et image.

## 6.1 Macros de Configuration

Macros PROGRESS_BAR_CONFIG_DEFAULT, SPINNER_CONFIG_DEFAULT, LABEL_CONFIG_DEFAULT, IMA

```
#define PROGRESS_BAR_CONFIG_DEFAULT ((progress_bar_config){ \
    .fraction = 0.0, \
    .text = NULL, \
    .orientation = GTK_ORIENTATION_HORIZONTAL, \
    .show_text = false, \
    .inverted = false, \
    .pulse_once = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})

#define SPINNER_CONFIG_DEFAULT ((spinner_config){ \
    .size = 0, \
    .spinning = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})

#define LABEL_CONFIG_DEFAULT ((label_config){ \
    .text = NULL, \
    .selectable = false, \
    .wrap = false, \
    .ellipsize = PANGO_ELLIPSIZE_NONE, \
    .max_width_chars = 0, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})

#define IMAGE_CONFIG_DEFAULT ((image_config){ \
    .icon_name = NULL, \
    .file_path = NULL, \
    .pixel_size = 0, \
    .keep_aspect = true, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})
```

Ces macros sécurisent l'initialisation des widgets d'affichage avec des valeurs neutres. Les pointeurs NULL empêchent les lectures de chaînes invalides, et les booléens explicites stabilisent le rendu par défaut. Cela limite les erreurs mémoire lors des premiers essais d'intégration.

Les macros d'affichage permettent également de standardiser le comportement visuel entre modules : une image sans chemin reste valide, une barre de progression sans texte reste fonctionnelle. Cette robustesse évite des plantages lors de données incomplètes.

## 6.2 Fonctions API

```c
GtkWidget *create_progress_bar(const progress_bar_config *config) {
    GtkWidget *progress = gtk_progress_bar_new();
    if (config == NULL) return progress;

    gtk_progress_bar_set_fraction(GTK_PROGRESS_BAR(progress), config->fraction);
    gtk_progress_bar_set_show_text(GTK_PROGRESS_BAR(progress), config->show_text
        );
    gtk_orientable_set_orientation(GTK_ORIENTABLE(progress), config->orientation
        );
    if (config->text != NULL) {
        gtk_progress_bar_set_text(GTK_PROGRESS_BAR(progress), config->text);
    }
    apply_widget_style(progress, &config->style);
    return progress;
}

GtkWidget *create_image(const image_config *config) {
    GtkWidget *widget;
    if (config == NULL) return gtk_image_new();

    if (config->file_path != NULL && config->file_path[0] != '\0') {
        GtkWidget *pic = gtk_picture_new_for_filename(config->file_path);
        gtk_picture_set_content_fit(GTK_PICTURE(pic),
            config->keep_aspect ? GTK_CONTENT_FIT_CONTAIN : GTK_CONTENT_FIT_FILL)
                ;
        widget = pic;
    } else {
        widget = gtk_image_new_from_icon_name(config->icon_name);
    }

    apply_widget_style(widget, &config->style);
    return widget;
}
```

Le module choisit dynamiquement la meilleure représentation visuelle (icône système ou image fichier) en conservant une API homogène.

La fonction `create_image` illustre bien l'approche du wrapper : masquer la complexité GTK derrière une interface simple. Le code applicatif choisit seulement la source (fichier ou icône) et le module gère le reste.

# 7 Module input

Le module input encapsule les widgets de saisie utilisateur : champ texte, liste déroulante et spin button.

## 7.1 Macros de Configuration

**Macros** ENTRY_CONFIG_DEFAULT, DROPDOWN_CONFIG_DEFAULT, SPIN_BUTTON_CONFIG_DEFAULT

```
#define ENTRY_CONFIG_DEFAULT ((entry_config){ \
    .placeholder = NULL, \
    .default_text = NULL, \
    .purpose_hint = NULL, \
    .password = false, \
    .editable = true, \
    .max_length = 0, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_changed = NULL, \
    .user_data = NULL, \
})

#define DROPDOWN_CONFIG_DEFAULT ((dropdown_config){ \
    .options = NULL, \
    .selected_index = 0, \
    .enable_search = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_selected = NULL, \
    .user_data = NULL, \
})

#define SPIN_BUTTON_CONFIG_DEFAULT ((spin_button_config){ \
    .min_value = 0.0, \
    .max_value = 100.0, \
    .step = 1.0, \
    .page = 5.0, \
    .value = 0.0, \
    .digits = 0, \
    .numeric_only = false, \
    .wrap = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_value_changed = NULL, \
    .user_data = NULL, \
})
```

Ces macros fixent une base stable pour les champs de saisie et évitent les paramètres incohérents. Les callbacks nuls et pointeurs NULL empêchent des appels invalides au runtime. C'est une protection simple mais efficace contre les segfaults en C.

L'initialisation par macros rend aussi le comportement de saisie reproductible entre écrans : même valeur par défaut, même politique de validation, même branchement des événements. Cela simplifie les tests et la maintenance.

## 7.2 Fonctions API

```c
GtkWidget *create_entry(const entry_config *config) {
    GtkWidget *entry = gtk_entry_new();
    if (config == NULL) return entry;

    if (config->placeholder) {
        gtk_entry_set_placeholder_text(GTK_ENTRY(entry), config->placeholder);
    }
    if (config->default_text) {
        gtk_editable_set_text(GTK_EDITABLE(entry), config->default_text);
    }

    gtk_entry_set_visibility(GTK_ENTRY(entry), !config->password);
    gtk_editable_set_editable(GTK_EDITABLE(entry), config->editable);
    apply_widget_style(entry, &config->style);

    if (config->on_changed) {
        g_signal_connect(entry, "changed", G_CALLBACK(config->on_changed), config
            ->user_data);
    }
    return entry;
}
```

La logique de création sépare clairement la configuration visuelle, le comportement de saisie et la connexion des événements utilisateur.

Le module `input` formalise ainsi un flux de données propre : saisie utilisateur, transformation, puis notification via signal. Cette séparation est utile pour concevoir une interface robuste et évolutive.

# 8 Module `toggle`

Le module `toggle` gère les contrôles binaires : case à cocher et interrupteur.

## 8.1 Macros de Configuration

**Macros `CHECKBOX_CONFIG_DEFAULT` et `SWITCH_CONFIG_DEFAULT`**

```
#define CHECKBOX_CONFIG_DEFAULT ((checkbox_config){ \
    .label = NULL, \
    .active = false, \
    .inconsistent = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_toggled = NULL, \
    .user_data = NULL, \
})

#define SWITCH_CONFIG_DEFAULT ((switch_config){ \
    .label = NULL, \
    .active = false, \
    .state = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_active_changed = NULL, \
    .user_data = NULL, \
})
```

Ces macros assurent une initialisation contrôlée des états booléens et des callbacks. En laissant `NULL` par défaut, on évite d'appeler des fonctions non initialisées. Cela limite les crashs lors des changements d'état.

Dans une application interactive, cette discipline d'initialisation est critique : un état binaire mal initialisé peut produire une interface incohérente et des bugs difficiles à diagnostiquer. Les macros réduisent ce risque.

## 8.2 Fonctions API

```
GtkWidget *create_switch_row(const switch_config *config, GtkWidget **out_switch
    ) {
    GtkWidget *row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    GtkWidget *label = gtk_label_new(config && config->label ? config->label : "
        ");
    GtkWidget *sw = gtk_switch_new();

    gtk_box_append(GTK_BOX(row), label);
    gtk_box_append(GTK_BOX(row), sw);

    if (config != NULL) {
        gtk_switch_set_active(GTK_SWITCH(sw), config->active);
        gtk_switch_set_state(GTK_SWITCH(sw), config->state);
        apply_widget_style(row, &config->style);
```

```
13
14        if (config->on_active_changed != NULL) {
15            g_signal_connect(sw, "notify::active", G_CALLBACK(config->
                  on_active_changed), config->user_data);
16        }
17    }
18
19    if (out_switch != NULL) {
20        *out_switch = sw;
21    }
22    return row;
23 }
```

Le module fournit des composants simples à intégrer tout en conservant un raccordement événementiel explicite.

`create_switch_row` encapsule la composition visuelle (label + switch) et la logique de notification. Le résultat est un composant réutilisable prêt à intégrer dans des formulaires de préférences.

# 9 Module date

Le module `date` encapsule `GtkCalendar` pour la sélection de dates.

## 9.1 Macros de Configuration

**Macro CALENDAR_CONFIG_DEFAULT**

```
#define CALENDAR_CONFIG_DEFAULT ((calendar_config){ \
    .selected_date = NULL, \
    .show_day_names = true, \
    .show_week_numbers = false, \
    .no_month_change = false, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
    .on_day_selected = NULL, \
    .user_data = NULL, \
})
```

Cette macro initialise le calendrier dans un état standard et sûr. Les callbacks et données utilisateur sont nuls tant qu'ils ne sont pas explicitement fournis. Cela évite les appels invalides pendant la navigation calendrier.

L'usage d'une macro par défaut facilite aussi la compatibilité entre versions GTK : la configuration de base reste valide même si certains réglages évoluent. Cela protège le code de haut niveau contre des regressions.

## 9.2 Fonctions API

```
GtkWidget *create_calendar(const calendar_config *config) {
    GtkWidget *calendar = gtk_calendar_new();

    if (config == NULL) {
        return calendar;
    }

    if (config->selected_date != NULL) {
#if GTK_CHECK_VERSION(4, 20, 0)
        gtk_calendar_set_date(GTK_CALENDAR(calendar), config->selected_date);
#else
        gtk_calendar_select_day(GTK_CALENDAR(calendar), config->selected_date);
#endif
    }

    gtk_calendar_set_show_day_names(GTK_CALENDAR(calendar), config->
        show_day_names);
    gtk_calendar_set_show_week_numbers(GTK_CALENDAR(calendar), config->
        show_week_numbers);
    gtk_calendar_set_show_heading(GTK_CALENDAR(calendar), !config->
        no_month_change);

```

```
    if (config->on_day_selected != NULL) {
        g_signal_connect(calendar, "day-selected", G_CALLBACK(config->
            on_day_selected), config->user_data);
    }

    return calendar;
}
```

La fonction gère proprement la compatibilité GTK selon la version disponible.

Elle centralise la sélection de date, la configuration d'affichage et le branchement du signal `day-selected`. Cette centralisation rend le comportement du widget calendrier plus prévisible dans toute l'application.

# 10 Module menu

Le module menu construit des menus hiérarchiques et enregistre les actions GTK associées.

## 10.1 Macros de Configuration

Macros MENU_ITEM_CONFIG_DEFAULT, MENU_SECTION_CONFIG_DEFAULT, MENUBAR_CONFIG_DEFAULT

```
#define MENU_ITEM_CONFIG_DEFAULT ((menu_item_config){ \
    .label = NULL, \
    .kind = MENU_ITEM_ACTION, \
    .children = NULL, \
    .child_count = 0, \
    .enabled = true, \
    .on_activate = NULL, \
    .user_data = NULL, \
})

#define MENU_SECTION_CONFIG_DEFAULT ((menu_section_config){ \
    .label = NULL, \
    .items = NULL, \
    .item_count = 0, \
})

#define MENUBAR_CONFIG_DEFAULT ((menubar_config){ \
    .sections = NULL, \
    .section_count = 0, \
    .action_map = NULL, \
    .layout = MENU_LAYOUT_HORIZONTAL, \
    .show_arrow_indicators = true, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})
```

Ces macros posent une base d'arbre de menu vide mais valide. Les compteurs à 0 et pointeurs NULL empêchent les parcours hors bornes et les déréférencements invalides. C'est particulièrement important dans ce module qui manipule la récursivité.

Ces valeurs par défaut sont aussi essentielles pour stabiliser l'algorithme récursif de construction des sous-menus. Une initialisation partielle peut facilement provoquer des accès mémoire invalides ; les macros réduisent ce risque.

## 10.2 Fonctions API

```
GtkWidget *create_menubar(const menubar_config *config) {
    GMenuModel *model = build_menu_model(config);
    GtkWidget *bar = gtk_popover_menu_bar_new_from_model(model);
    if (config != NULL) {
        apply_widget_style(bar, &config->style);
    }
```

```
    g_object_unref(model);
    return bar;
}

GtkWidget *create_menu_panel(const menubar_config *config) {
    GMenuModel *model = build_menu_model(config);
    GtkWidget *panel = gtk_popover_menu_new_from_model(model);
    g_object_unref(model);
    return panel;
}
```

Le module sépare la description logique du menu (structures) de la génération GTK (`GMenuModel` + widgets).

La fonction `create_menubar` sert de point d'entrée unique : elle construit le modèle puis instancie le widget final. Cette séparation améliore la lisibilité et facilite l'ajout de nouvelles actions.

# 11 Module dialog

Le module dialog fournit une API de boîtes d'alerte synchrones ou asynchrones basée sur GtkAlertDialog.

## 11.1 Macros de Configuration

**Macros ALERT_BUTTON_CONFIG_DEFAULT et ALERT_DIALOG_CONFIG_DEFAULT**

```
#define ALERT_BUTTON_CONFIG_DEFAULT ((alert_button_config){ \
    .label = NULL, \
    .appearance = NULL, \
})

#define ALERT_DIALOG_CONFIG_DEFAULT ((alert_dialog_config){ \
    .parent = NULL, \
    .message = NULL, \
    .detail = NULL, \
    .buttons = NULL, \
    .button_count = 0, \
    .default_button = -1, \
    .cancel_button = -1, \
    .modal = true, \
    .on_response = NULL, \
    .user_data = NULL, \
})
```

Les macros définissent une alerte sûre par défaut, avec des index invalides explicites pour les boutons. Les pointeurs NULL empêchent les accès à des chaînes ou callbacks non initialisés. Cette approche défensive réduit les risques de segmentation fault dans les chemins d'interaction utilisateur.

L'usage de valeurs sentinelles (-1) pour default_button et cancel_button évite les index hors limites. C'est une pratique robuste pour prévenir les erreurs de sélection côté UI.

## 11.2 Fonctions API

```
void show_alert_dialog(const alert_dialog_config *config) {
    GtkAlertDialog *dialog;

    if (config == NULL || config->message == NULL) {
        return;
    }

    dialog = gtk_alert_dialog_new("%s", config->message);
    if (config->detail != NULL) {
        gtk_alert_dialog_set_detail(dialog, config->detail);
    }

```

```
    if (config->on_response != NULL) {
        gtk_alert_dialog_choose(dialog, config->parent, NULL, on_alert_response,
            ctx);
    } else {
        gtk_alert_dialog_show(dialog, config->parent);
    }

    g_object_unref(dialog);
}
```

La fonction choisit automatiquement un mode asynchrone si un callback est fourni, sinon un mode simple d'affichage.

Ce design permet de couvrir deux besoins : une alerte informative rapide et une alerte interactive qui remonte un résultat utilisateur. L'API reste compacte tout en masquant la complexité asynchrone de GTK.

# 12 Module separator

Le module separator crée une séparation visuelle horizontale ou verticale entre zones d'interface.

## 12.1 Macros de Configuration

**Macro SEPARATOR_CONFIG_DEFAULT**

```
#define SEPARATOR_CONFIG_DEFAULT ((separator_config){ \
    .orientation = GTK_ORIENTATION_HORIZONTAL, \
    .style = WIDGET_STYLE_CONFIG_DEFAULT, \
})
```

Cette macro initialise un séparateur prêt à l'emploi dans le cas le plus fréquent (horizontal). Elle évite d'avoir une orientation non définie au moment du rendu. La valeur de style par défaut garantit aussi une application cohérente du thème.

Même pour un composant simple, une initialisation explicite reste importante : elle supprime les ambiguïtés d'affichage et évite des anomalies d'orientation dans des layouts complexes.

## 12.2 Fonctions API

```
GtkWidget *create_separator(const separator_config *config) {
    GtkWidget *separator = gtk_separator_new(
        config ? config->orientation : GTK_ORIENTATION_HORIZONTAL
    );
    if (config != NULL) {
        apply_widget_style(separator, &config->style);
    }
    return separator;
}
```

La fonction conserve un comportement stable même sans configuration explicite.

Elle constitue un bon exemple de wrapper minimal : une API courte, des valeurs par défaut robustes et un point unique de personnalisation via apply_widget_style.

# 13 Module theme

Le module theme centralise l'application du thème CSS de démonstration.

## 13.1 Macros de Configuration

**Aucune macro #define de configuration publique**

Le fichier d'en-tête theme.h expose uniquement la fonction apply_demo_theme. Il n'y a pas de structure *_config ni de macro *_CONFIG_DEFAULT dans ce module.

Le module theme est volontairement centré sur le comportement global de rendu plutôt que sur des instances de configuration locales. Cette approche évite la dispersion des styles dans plusieurs composants.

## 13.2 Fonctions API

```c
void apply_demo_theme(void) {
    GtkCssProvider *provider = gtk_css_provider_new();
    const char *css =
        ".btn-primary {"
        " background: linear-gradient(180deg, #1f8c7a, #176b5e);"
        " color: #ffffff;"
        "}"
        ".btn-success {"
        " background: linear-gradient(180deg, #3c8f28, #2f6f1f);"
        " color: #ffffff;"
        "}"
        ".btn-danger {"
        " background: linear-gradient(180deg, #b74040, #8e2f2f);"
        " color: #ffffff;"
        "}";

    gtk_css_provider_load_from_string(provider, css);
    gtk_style_context_add_provider_for_display(
        gdk_display_get_default(),
        GTK_STYLE_PROVIDER(provider),
        GTK_STYLE_PROVIDER_PRIORITY_APPLICATION
    );
    g_object_unref(provider);
}
```

Ce module injecte des classes CSS applicatives, ensuite utilisées par les widgets (ex : boutons primary/success/danger).

La fonction apply_demo_theme centralise les règles visuelles et garantit une cohérence graphique sur toute l'application. Elle réduit aussi le risque d'erreurs liées à des styles CSS dupliqués.

# 14 Module demo_app

Le module demo_app assemble les widgets du projet et lance l'interface de démonstration.

## 14.1 Macros de Configuration

**Aucune macro de configuration dédiée dans demo_app.h**

Le header demo_app.h expose uniquement la fonction de démarrage demo_app_start. Les configurations utilisées proviennent des modules widgets et de leurs macros *_CONFIG_DEFAULT.

Le rôle de ce module est d'orchestrer les configurations existantes plutôt que de définir de nouvelles macros. Il sert de point d'intégration pour démontrer l'usage combiné des composants.

## 14.2 Fonctions API

```c
void demo_app_start(GtkApplication *app, gpointer user_data) {
    (void)user_data;
    static bool once = false;
    if (!once) {
        apply_demo_theme();
        once = true;
    }

    GtkWidget *window = create_window(app, &(window_config){
        .title = "GTK UI Wrapper Showcasea",
        .icon_name = "applications-development-symbolic",
        .widget_name = "wrapper-window",
        .background_image_path = NULL,
        .default_width = 700,
        .default_height = 300,
        .present_on_create = false,
    });

    showcase_state *state = g_new0(showcase_state, 1);
    gtk_window_set_child(GTK_WINDOW(window),
                         build_showcase(app, GTK_WINDOW(window), state));
    gtk_window_present(GTK_WINDOW(window));
}
```

Cette fonction initialise le thème une seule fois puis compose la fenêtre principale avec l'ensemble des pages de démonstration.

demo_app_start est la passerelle entre l'initialisation applicative GTK et le système de widgets du projet. Elle montre un exemple complet de composition d'interface basé sur les wrappers.